CPSC 313: Computer Hardware and Operating Systems

Unit 2: Pipelining

Intro and Overview; Analyzing pipeline timing



The 2nd Edition

CS Student Research Conference

November 6th @ ICCS Atrium

Submission tracks open!

- Short Paper + Oral Presentation Track
- Poster + Poster Presentation Track

Timeline

- Sep 18: Registration (title + abstract due)
- Sep 25: Full Paper/Poster Submission
- Oct 9: Notifications
- Oct 23: Camera-Ready Deadline
- Nov 6: Conference Day!

Program at a Glance

- Keynote by Dr. Hila Gonen
- Poster Session
- Student Oral Presentations
- Lunch & Networking
- Junior & Senior Workshop Sessions
- Career Panel
- Awards!

Check out our website for more details!
ubccsss.org/cssrc

Admin

- Quiz 1 running this week
  - Please refrain from questions, including on Piazza, while the quiz is live
  - The quiz will be live from Tuesday through Saturday
- Tutorial 3 this week

Admin (Lab 3)

- Was released last Friday
- Good news:
  - You can work with a partner for part of it.
- Less good news:
  - You are still responsible for understanding all the code that you and your partner submit.

Admin (Lab 3)

- Lab 3 is released as two separate parts:
  - Lab3-Ind: These are a set of questions about the code and assignment designed to prepare you to undertake the coding portion. Worth ¼ of the lab.
  - Lab3-Partner: This is the larger part and can be completed with a partner. Worth ¾ of the lab.
- It is significantly more difficult than labs 1 and 2.

Introduction to pi(e)pelined CPU architectures

- The key observation is that most of the time, most of the circuitry in our sequential CPU is idle.
- We want to avoid so much idle hardware

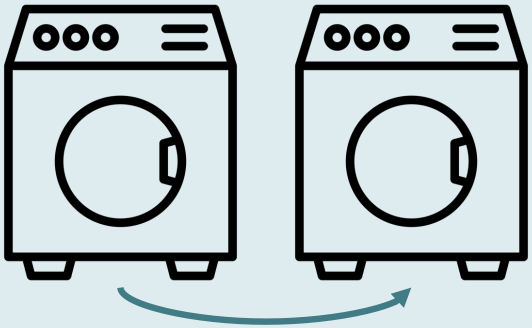
Please do Norm's laundry
Which is better?



Regular Laundry World

Washer:
40 mins

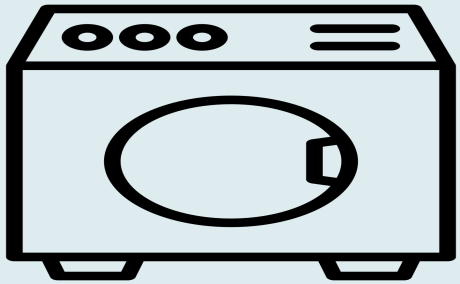
Dryer:
60 mins



time to change: 20 minutes
(Norm is reaaally sloooow)

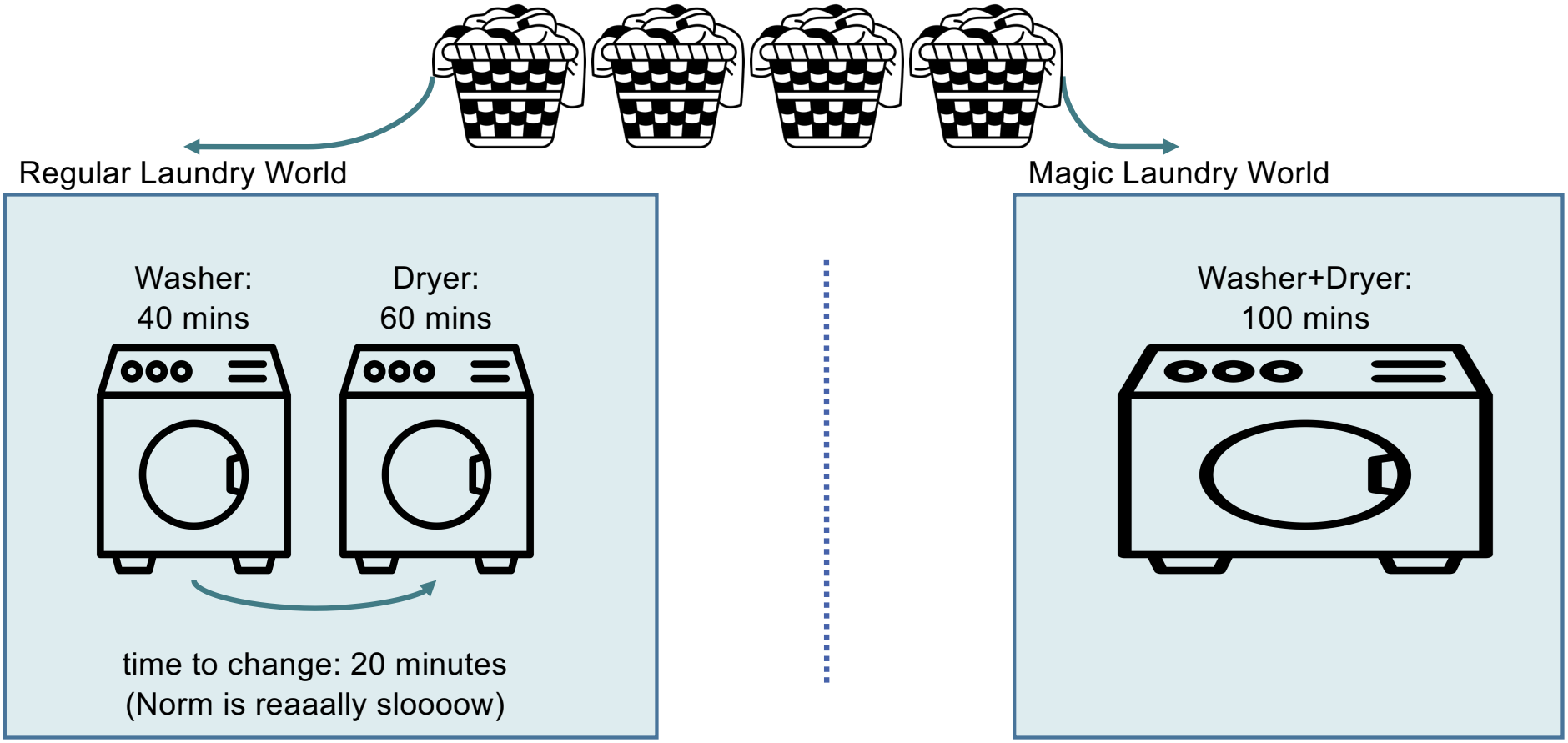
Magic Laundry World

Magical
Washer+Dryer:
100 mins



(such a thing could not
truly exist in mundane reality)

Ha! Norm has kids. *Now* do Norm's laundry
Which is better?



Terminology we'll discuss now

- **Stage**:
  - What we used to call "phase".*
  - When we talk about pipelines, each piece of the pipeline is a stage.
  - A stage is the unit of computation that the processor performs in a single clock cycle.

* We will screw up this terminology regularly. It's there just to remind us there's a difference, not to either have you test us or us test you on it. Be forgiving!

- Instruction statuses:
  - **In-flight**: somewhere in the pipeline
  - **Retired**: instruction has completed execution
  - Go back to the laundry metaphor

Terminology we'll discuss later

- **Pipeline Register**:
  - Colloquially: the extra work we incur by pipelining.
  - In reality: a register in the processor that captures the set of signals (values) on which a stage will execute.
  - Each stage has its own pipeline register.
- **Hazard**:
  - The "problems" that we run into when we naively add pipelining

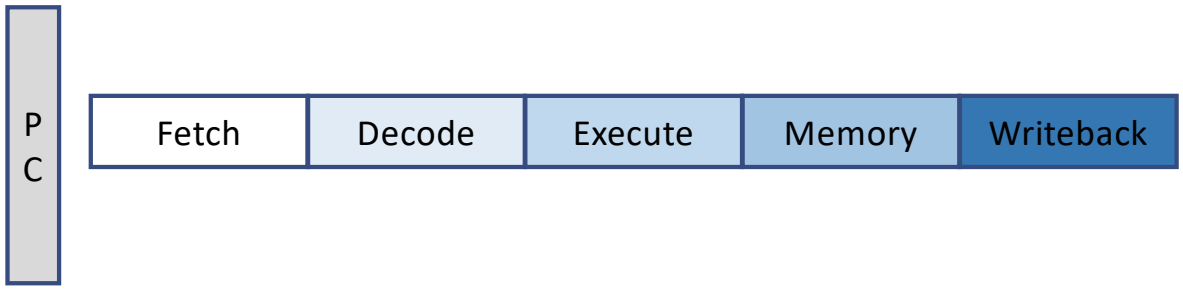
Where we're going in this module

- Module Map:
  - P6 Introduction to pipelining.
  - **2.1 Analyzing pipeline timing.**
  - P7 Pipeline registers.
  - 2.2 Handling hazards by inserting nop instructions.
  - P8 Handling hazards in hardware by stalling.
  - 2.3 Handling hazards in hardware by forwarding values.
  - P9 Control hazards

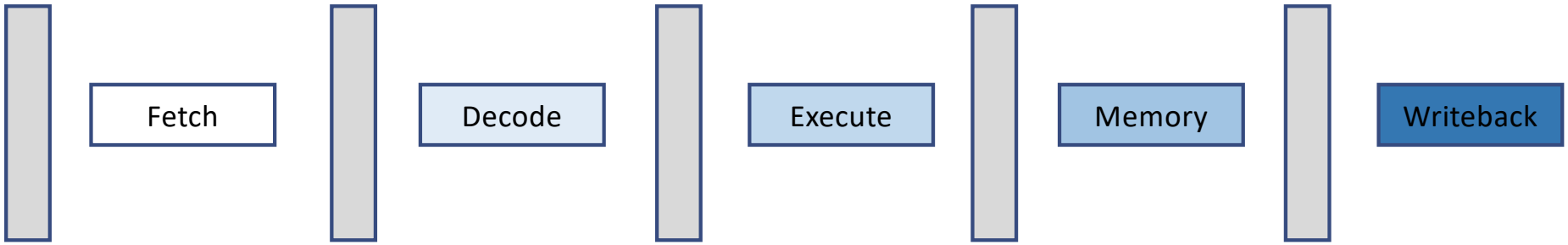
Where we're going in this module

- Module Map:
  - 2.4 Handling control hazards via branch prediction.
  - P10 Dealing with mispredictions.
  - 2.5 Evaluating pipeline performance.
  - P11 Pipeline optimization.
  - 2.6 Other forms of parallelism.

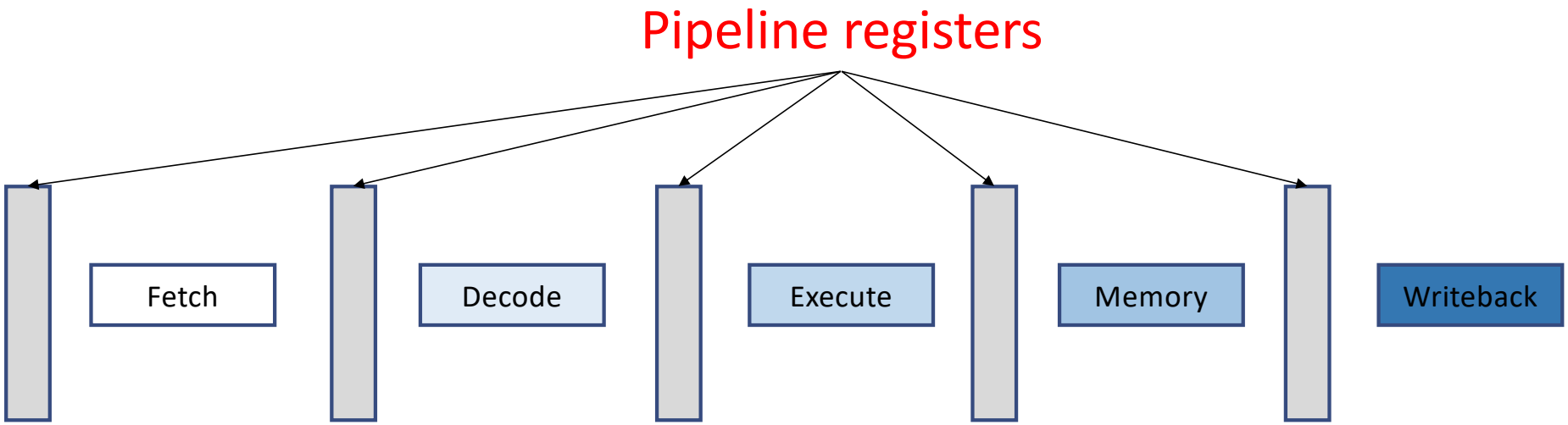
Before we had pipelining:



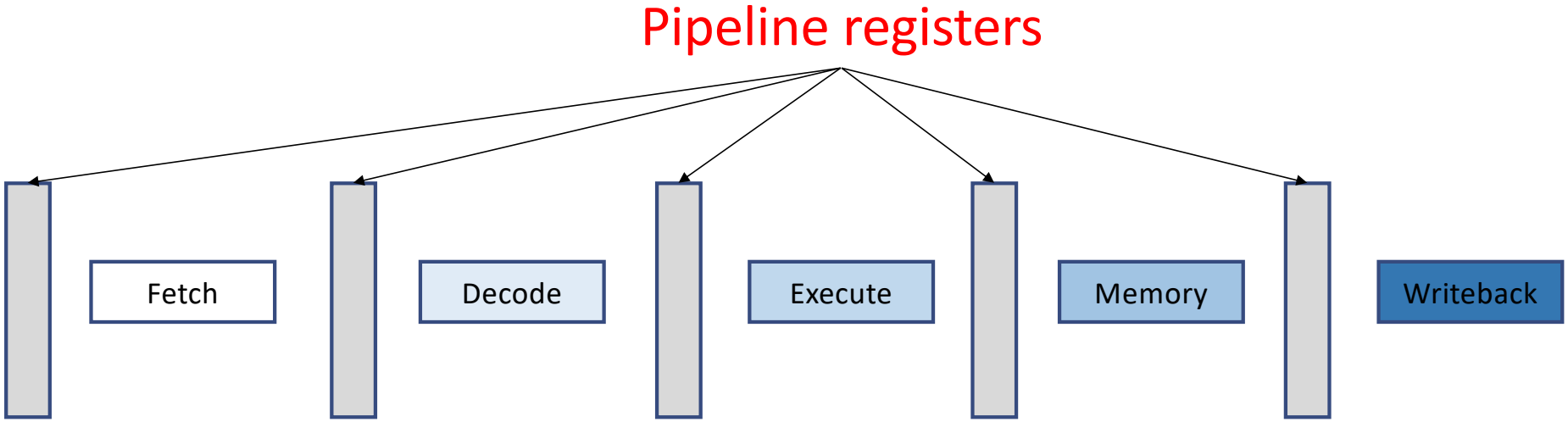
With Pipelining



With Pipelining



With Pipelining

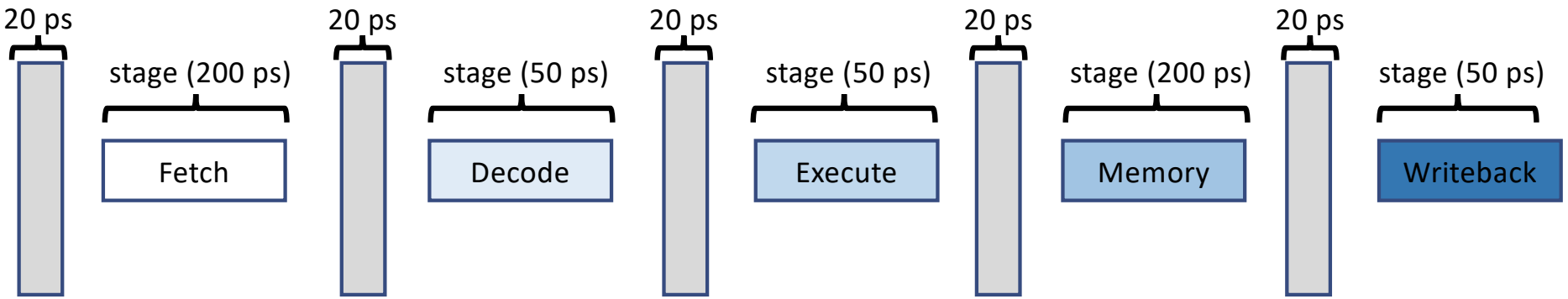


- Why do we need pipeline registers?
  - Each stage is working on a **different** instruction during the clock cycle.
  - We don't want one stage's output to affect the computations of the next stage within the same clock cycle.

Performance: Latency

Assume that:

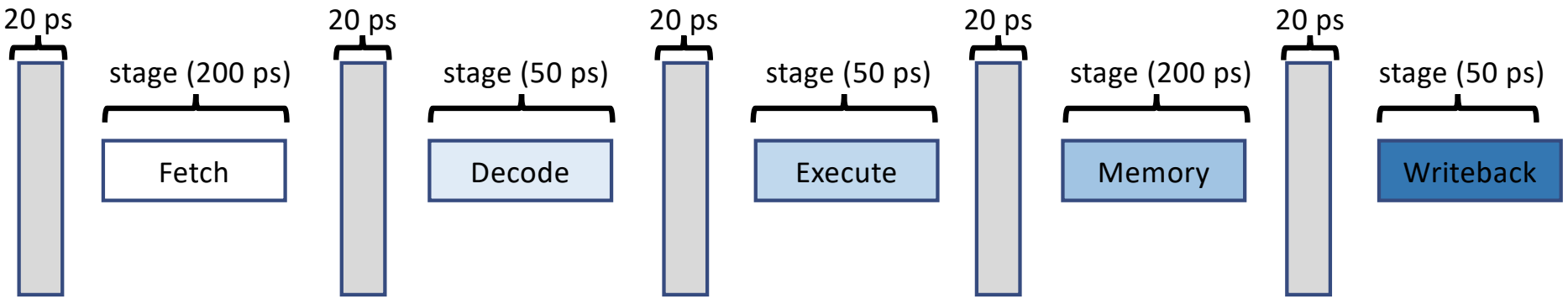
- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



Performance: Latency

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

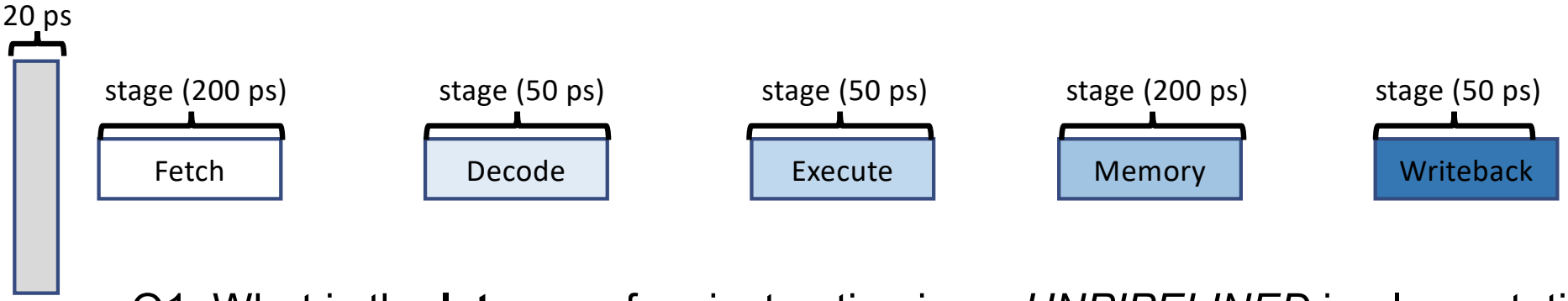


Latency: End to end time to complete execution of a single instruction.

Performance: Latency (1)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

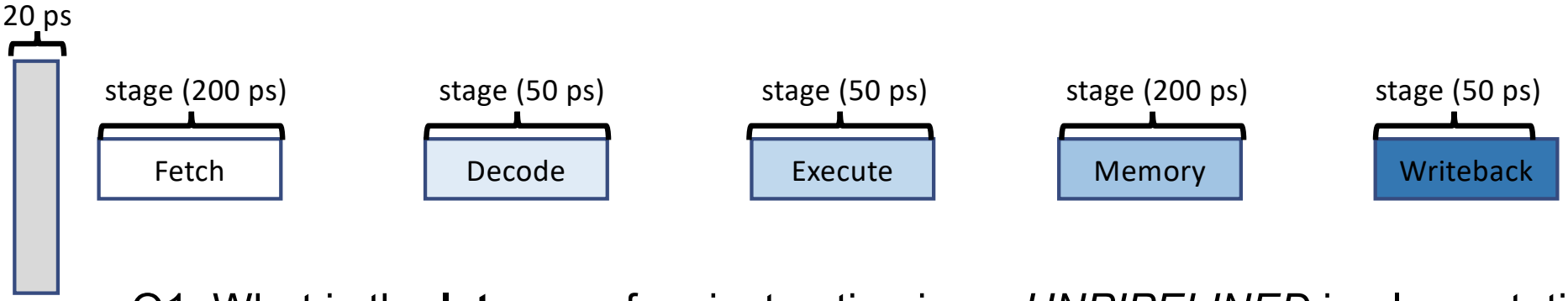


Q1: What is the **latency** of an instruction in an *UNPIPELINED* implementation?

Performance: Latency (1)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



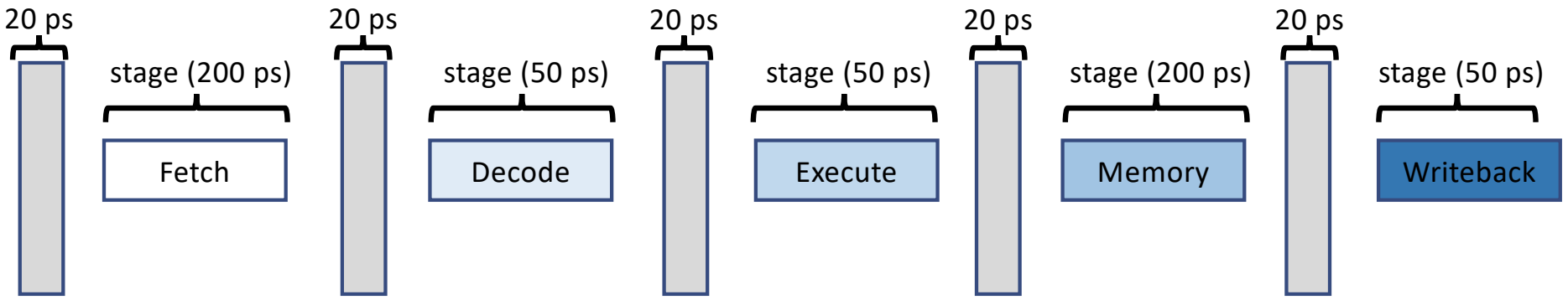
Q1: What is the **latency** of an instruction in an *UNPIPELINED* implementation?

20 + 200 + 50 + 50 + 200 + 50 = 570 ps

Performance: Latency (2)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



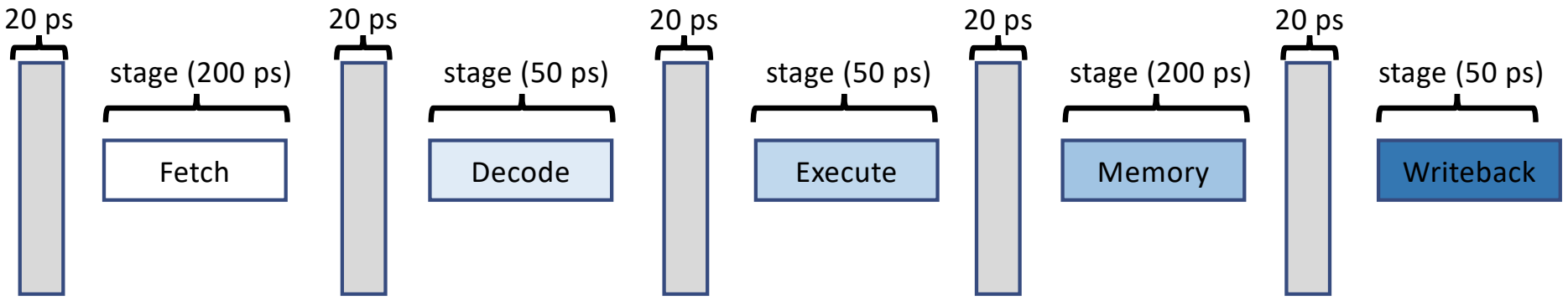
Q2: What is the **latency** of an instruction in a *PIPELINED* implementation?

Be careful! We couldn't move clothes from the washer to the dryer after 40 minutes. Why not?

Performance: Latency (2)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



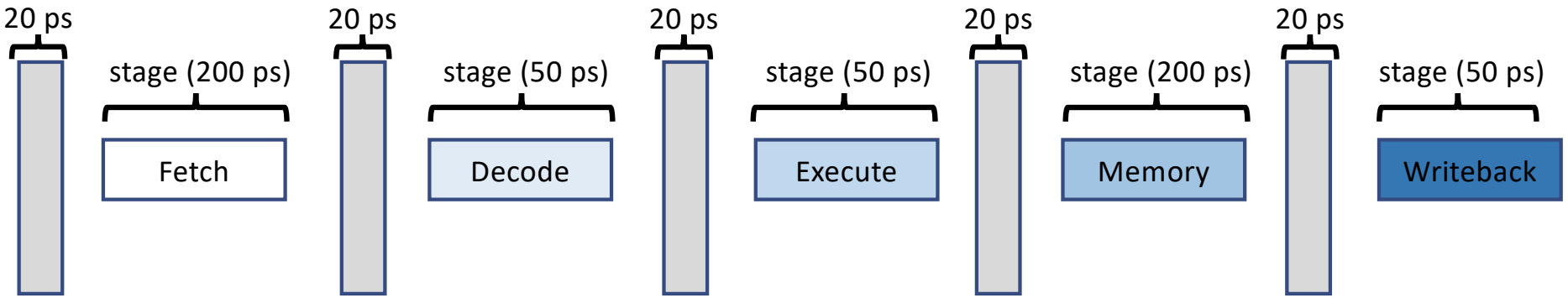
Q2: What is the **latency** of an instruction in a *PIPELINED* implementation?

200 * 5 + 20 * 5 = (200 + 20) * 5 = 1100 ps

Performance: Retirement Latency

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

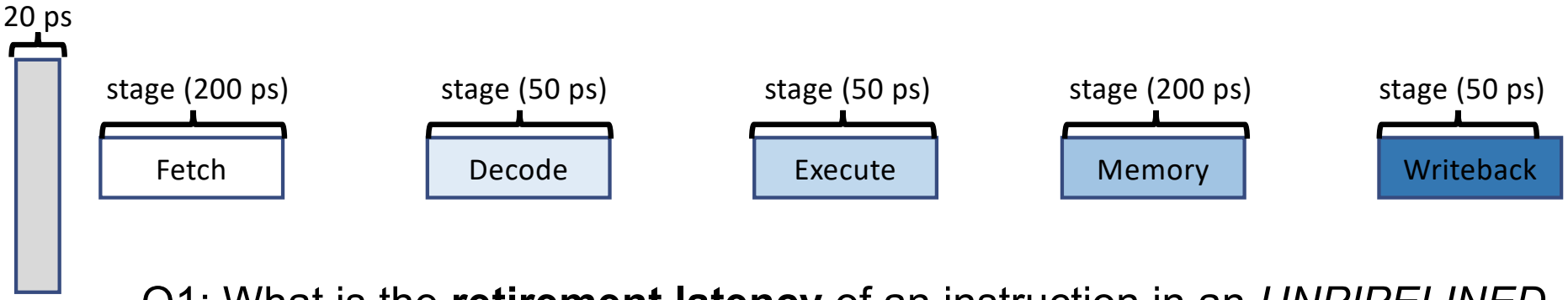


Retirement Latency: Time between the completion of one instruction and the completion of the next instruction.

Performance: Retirement Latency (1)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

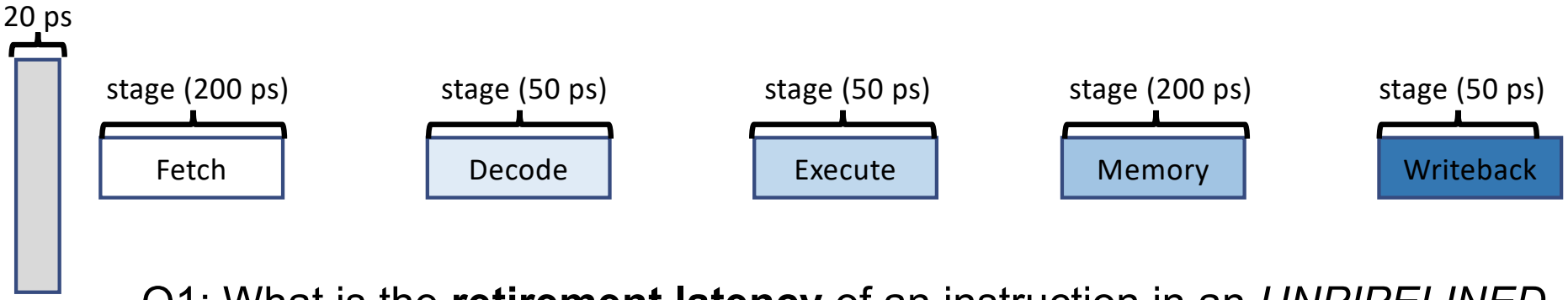


Q1: What is the **retirement latency** of an instruction in an *UNPIPELINED* implementation?

Performance: Retirement Latency (1)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



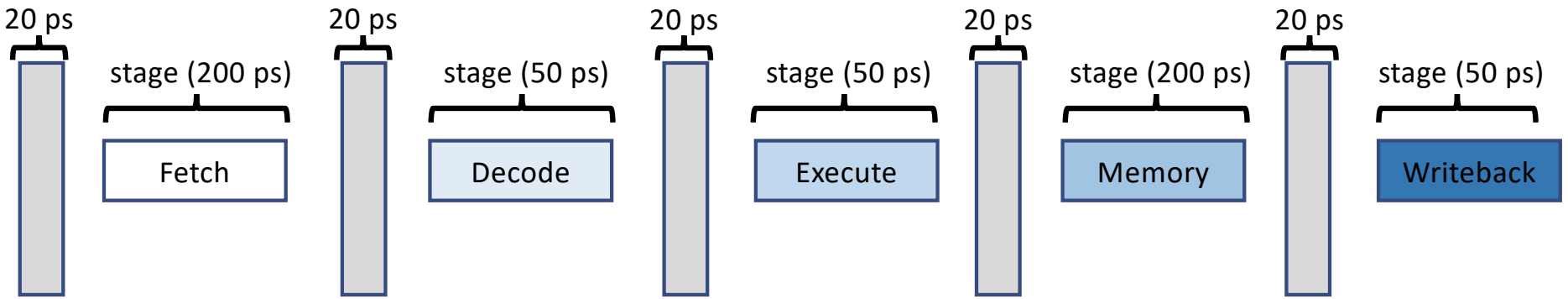
Q1: What is the **retirement latency** of an instruction in an *UNPIPELINED* implementation?

20 + 200 + 50 + 50 + 200 + 50 = 570 ps

Performance: Retirement Latency (2)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



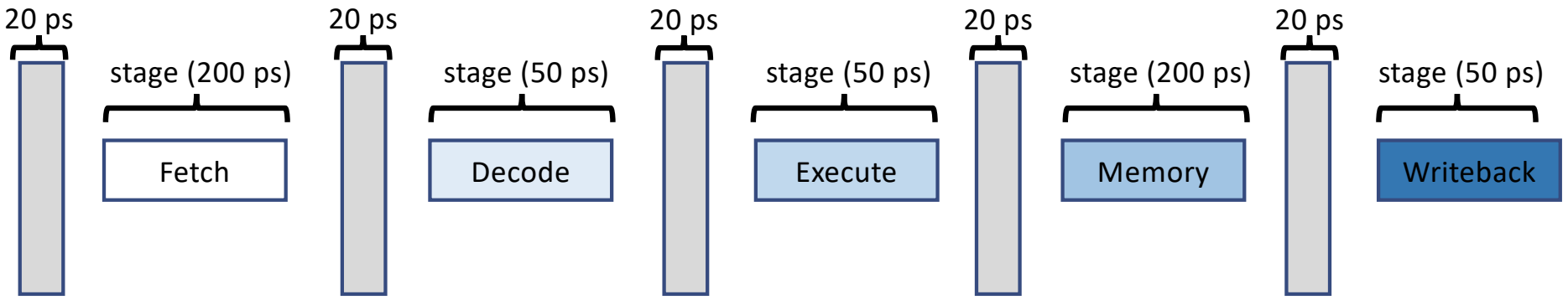
Q2: What is the **retirement latency** of an instruction in a *PIPELINED* implementation?

Q2: What is the **retirement latency** of an instruction in a *PIPELINED* implementation?

Performance: Retirement Latency (2)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



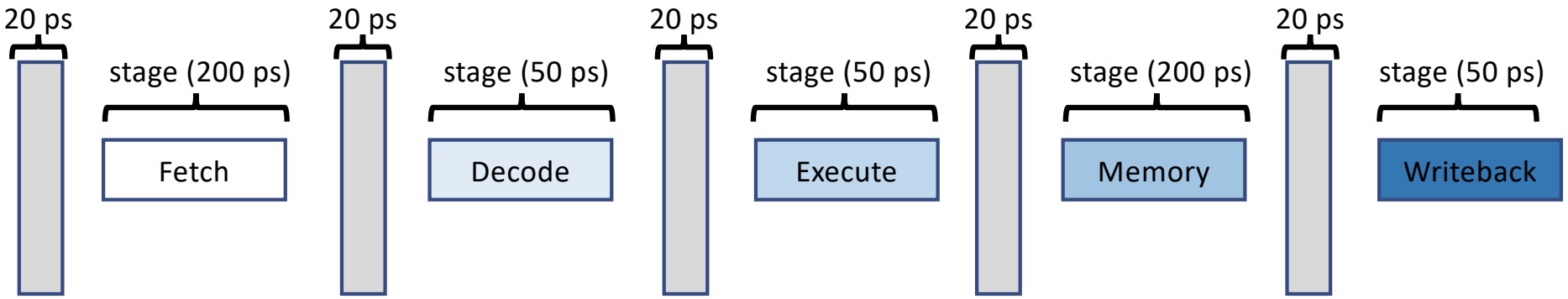
Q2: What is the **retirement latency** of an instruction in a *PIPELINED* implementation?

20 + 200 = 220 ps

Performance: Throughput

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

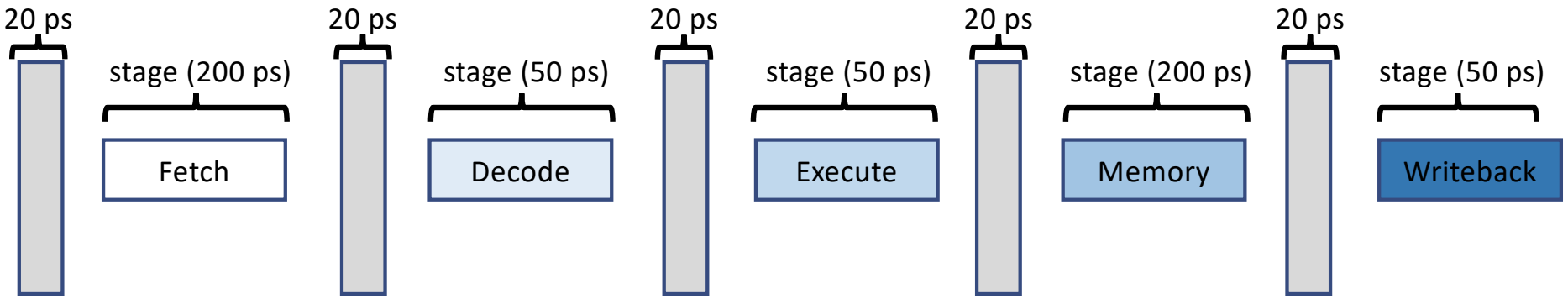


Throughput: The rate at which instructions complete. Typically expressed in GIPS: giga-instructions per second.

Performance: Throughput

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



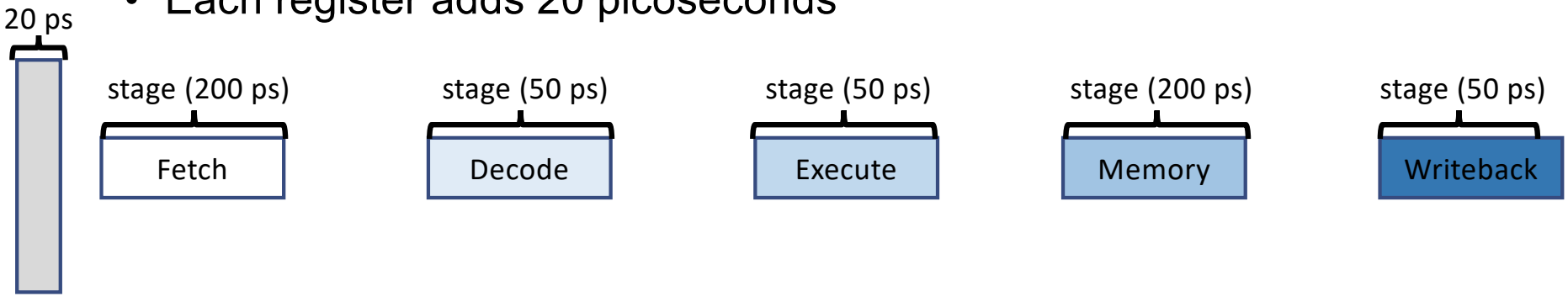
Throughput: The rate at which instructions complete. Typically expressed in GIPS: giga-instructions per second.

Note: We've computed picoseconds per instruction; throughput is instructions per second …

Performance: Throughput (1)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

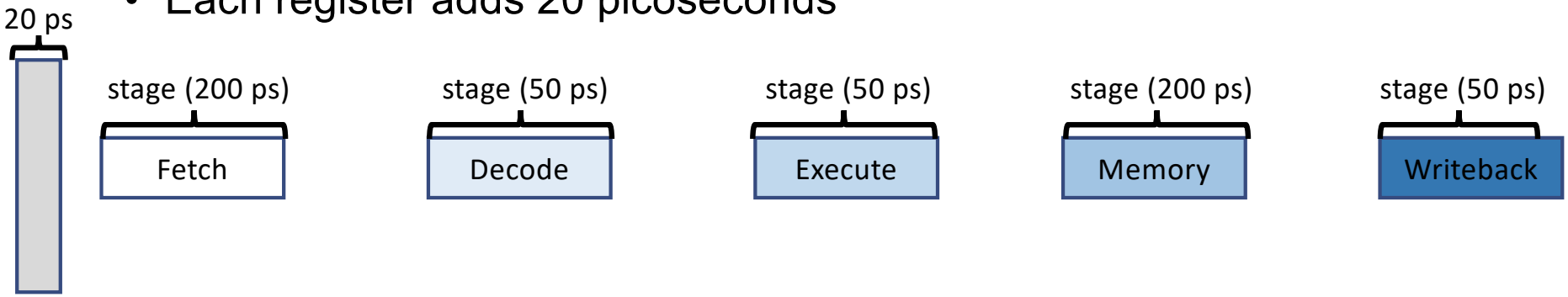


Q1: What is the **throughput** of an *UNPIPELINED* implementation?

Performance: Throughput (1)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



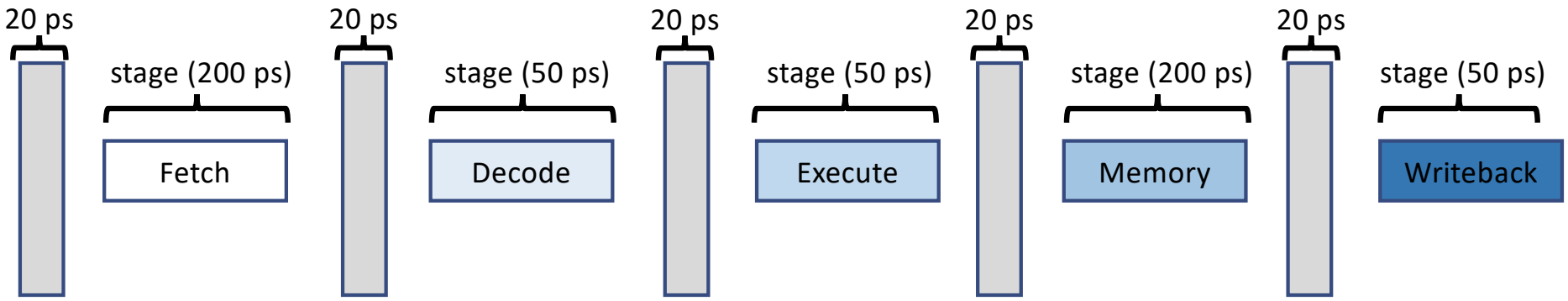
Q1: What is the **throughput** of an *UNPIPELINED* implementation?

1 / 570 ps per instruction = 1 / (570×10^{-12})s per instruction =
($1 / 570$) x 10^{12} IPS = ($1000 / 570$) x 10^{9} IPS = ($1000 / 570$) GIPS ≈ 1.75 GIPS

Performance: Throughput (2)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds

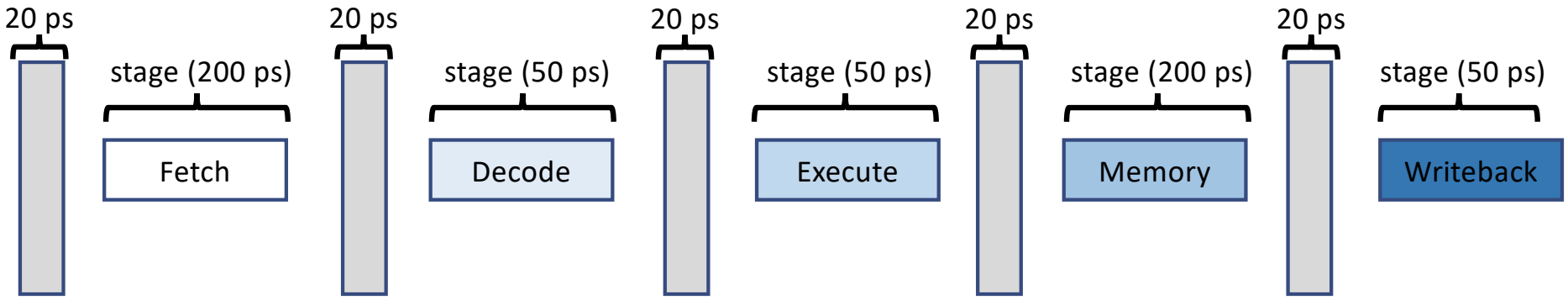


Q2: What is the **throughput** of a *PIPELINED* implementation?

Performance: Throughput (2)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



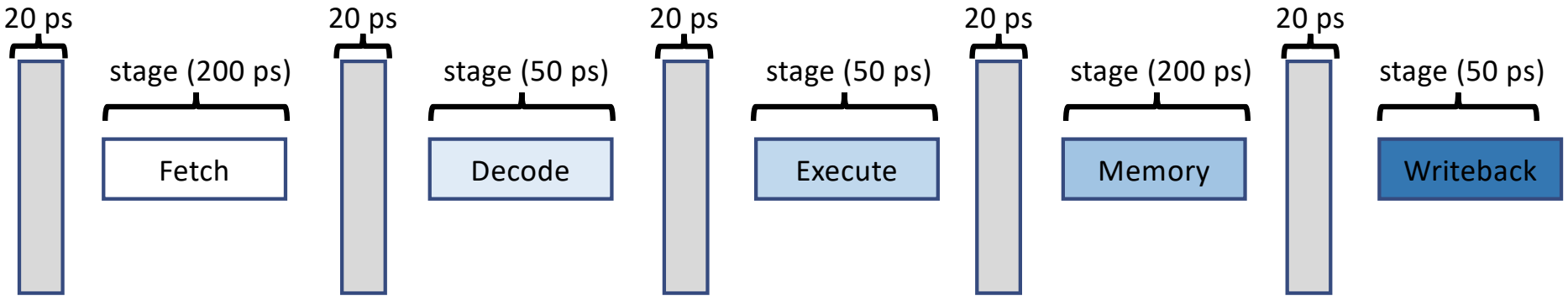
Q2: What is the **throughput** of a *PIPELINED* implementation?

1 / 220 ps per instruction = 1 / (220 x 10^{-12}) s per instruction =
(1 / 220) x 10^{12} IPS = (1000 / 220) x 10^{9} IPS = (1000 / 220) GIPS ≈ 4.55 GIPS

Performance: Throughput (3)

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



Q1: What is the **throughput** of an *UNPIPELINED* implementation?

1.75 GIPS

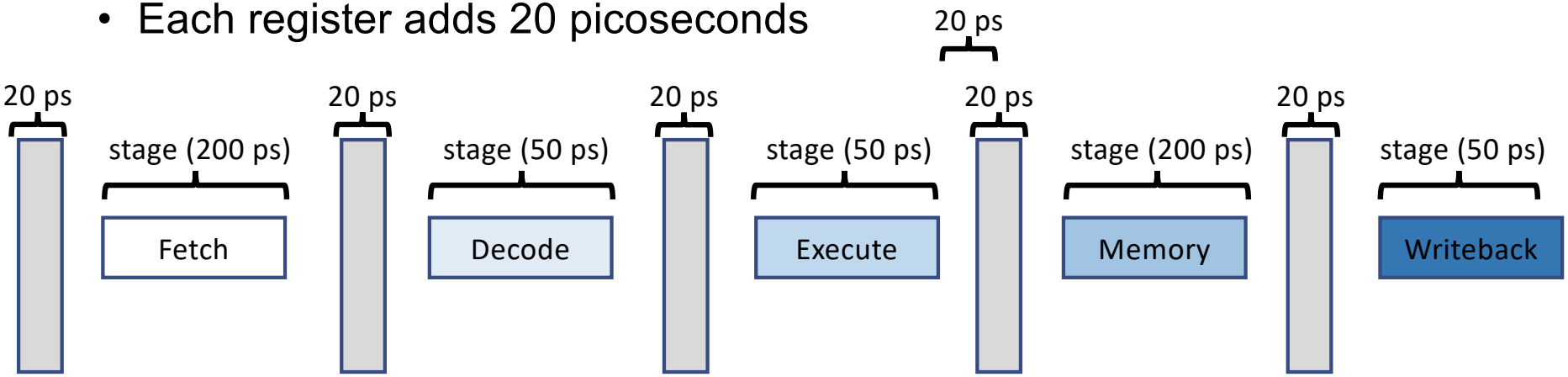
Q2: What is the **throughput** of a *PIPELINED* implementation?

4.55 GIPS

How fast is our pipelined processor?

Assume that:

- Each stage needs either 50 or 200 picoseconds to execute
- Each register adds 20 picoseconds



We increased latency by 93% but multiplied throughput by a factor of 2.6.

Pipeline Issues

- For a pipelined processor, throughput is the inverse of the retirement latency.
- Why don’t we just have thousands of really short stages to get maximum performance?

Pipeline Issues

- For a pipelined processor, throughput is the inverse of the retirement latency.
- Why don't we just have thousands of really short stages to get maximum performance?
  - The hardware might not partition well into too many pieces.
  - Each stage introduces a pipeline register that will add latency (and take up space on the chip and consume power and…)
  - Dealing with hazards (next class) becomes more complicated as the number of stages increases.

Pipeline length (depth) trivia (Intel)

Year	Microarchitecture	Pipeline stages
2015	Skylake	14
2018	Palm Cove	14
2019	Sunny Cove	12
2021	Cypress Cove	12
2022	Raptor Cove	12
2024	Lion Cove	10

- Min: 8086 (1978) 2 stages
- Max: Pentium 4 (2004) 31

In class activity

- PrairieLearn Inclass exercise 7 – Pipeline Performance

Takeaways

- Pipelining
  - Introduces parallelism
  - Allows multiple instructions to be executing at the same time
  - **Increases** single instruction latency
  - **Decreases** retirement latency
  - **Increases** throughput